

Project Completion Certificate

This is to certify that the Project work done at **C Institute** BY Mr./Ms. _____
(NIELIT Registration No. _____) in partial fulfillment of NIELIT 'O' Level Examination has
been found satisfactory.

This report has not been submitted for any other examination and does not form part of any other
course undergone by the candidate. It is further certifies that he/she has appeared in all the four
modules of NIELIT 'O' level Examination.

Head of the Organization:

Tarun Verma

Organization Name and Address:

C Institute

A-10, UIT Complex,

M.P. Colony,

Bikaner, Rajasthan

Name of the Guide/Supervisor:

Tarun Verma

Qualification :

A Level (NIELIT)

MSC Computer Science (MGSU Bikaner)

Fees Details: -

Payment Date	UTR Number	Account Holder Name

ACKNOWLEDGEMENT

This project has been completed as a part of the NIELIT 'O' Level Course.

I would like to express my sincere gratitude to my guide, Mr. Tarun Verma, C Institute, Bikaner, who gave me the golden opportunity to work on this wonderful project on the topic “Audio Noise Remover from Video”, which also helped me in doing a great deal of research and learning about audio signal processing, Python GUI development, and multimedia handling.

I am thankful for his invaluable guidance and constant assistance, without which the successful completion of this project would not have been possible. I also thank him for giving me the opportunity to explore real-world applications of programming and to understand how theoretical knowledge can be applied practically.

Finally, I would also like to thank my parents and friends who supported and helped me in completing this project within the specified time limit.

Alok Sahni
NIELIT 'O' Level Candidate

Project Report

Title: Audio Noise Remover from Video

By: Alok Sahni

1. Problem Statement

In many real-life situations such as interviews, podcasts, classroom recordings, or webinars, audio embedded in videos often contains background noise. This affects the clarity and professionalism of the content. The objective of this project is to extract the audio from a video file, remove background noise, and re-integrate the cleaned audio into the original video — using Python programming.

2. Objectives

- To build a GUI-based application in Python to remove noise from audio embedded in videos.
- To extract audio from .mp4 or .mkv files.
- To reduce noise using signal processing.
- To save and play both original and cleaned audio for comparison.
- To merge cleaned audio back into the video using FFmpeg.
- To demonstrate the use of external libraries and system utilities in a Python project.

3. Tools Used

Tool/Library	Purpose
Python 3.x	Programming Language
Tkinter	GUI Interface
moviepy	Extracting audio from video
scipy.io.wavfile	Reading/writing audio in WAV format
noisereduce	Reducing noise from audio
playsound	Audio preview
tempfile	Handling temporary files
subprocess	Running FFmpeg commands
FFmpeg	Replacing audio in video

4. System Requirements

- Operating System: Windows 10 or above
- Python 3.7+
- Required Libraries: moviepy, scipy, noisereduce, playsound, tkinter (usually built-in)
- External Tool: FFmpeg (added to system PATH or given absolute path)

5. Source Code

जरूरी libraries import की जा रही हैं

import os

import tkinter as tk

from tkinter import filedialog, messagebox

from tkinter.ttk import Progressbar

from moviepy.editor import VideoFileClip

from scipy.io import wavfile

import noisereduce as nr

import tempfile

import shutil

import subprocess

from playsound import playsound

Noise Remover Application की main class

class NoiseRemoverApp:

def __init__(self, root):

self.root = root

self.root.title("Audio Noise Remover from Video By Tarun Verma")

self.root.geometry("500x400") # window की size set कर रहे हैं

self.video_path = "" # user द्वारा select किया गया video path

self.cleaned_audio_path = "" # noise remove की गई audio का path

Label दिखा रहा है user को क्या करना है

self.label = tk.Label(root, text="Select a video file (.mp4/.mkv)", font=("Arial", 12))

```
self.label.pack(pady=10)
```

```
# File browse करने वाला button
```

```
self.select_button = tk.Button(root, text="Browse", command=self.browse_file)
```

```
self.select_button.pack(pady=5)
```

```
# Progress bar user को processing का status दिखाती है
```

```
self.progress = Progressbar(root, orient=tk.HORIZONTAL, length=300, mode='determinate')
```

```
self.progress.pack(pady=10)
```

```
# Audio clean करने का button
```

```
self.clean_button = tk.Button(root, text="Clean Audio", command=self.clean_audio)
```

```
self.clean_button.pack(pady=5)
```

```
# Original audio सुनने का button
```

```
self.preview_original_button = tk.Button(root, text="Preview Original Audio",  
command=self.preview_original_audio)
```

```
self.preview_original_button.pack(pady=5)
```

```
# Cleaned audio सुनने का button
```

```
self.preview_cleaned_button = tk.Button(root, text="Preview Cleaned Audio",  
command=self.preview_cleaned_audio)
```

```
self.preview_cleaned_button.pack(pady=5)
```

```
# Browse button की functionality - video file select करना
```

```
def browse_file(self):
```

```
    filetypes = [("Video files", "*.mp4 *.mkv")]
```

```
self.video_path = filedialog.askopenfilename(title="Open Video File", filetypes=filetypes)

if self.video_path:

    messagebox.showinfo("Selected", f"File selected:\n{self.video_path}")


# Original audio को सुनने की functionality

def preview_original_audio(self):

    if not self.video_path:

        messagebox.showwarning("No file", "Please select a video file first.")

        return


# Temporary file बन रही है जिसमें audio save होगा

temp_audio = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")


# Video से audio निकालना

clip = VideoFileClip(self.video_path)

clip.audio.write_audiofile(temp_audio.name, verbose=False, logger=None)

clip.close()


# Original audio play करना

playsound(temp_audio.name)

temp_audio.close()


# Audio file delete करना

os.unlink(temp_audio.name)


# Cleaned audio को सुनने की functionality
```

```
def preview_cleaned_audio(self):  
    if not self.cleaned_audio_path:  
        messagebox.showwarning("Not cleaned", "Please clean the audio first.")  
        return  
    playsound(self.cleaned_audio_path)  
  
# Audio clean करने वाली main functionality  
def clean_audio(self):  
    if not self.video_path:  
        messagebox.showwarning("No file", "Please select a video file first.")  
        return  
  
    self.progress['value'] = 10 # progress शुरू की जा रही है  
    self.root.update_idletasks()  
  
    temp_audio_path = None # temp audio path को बाद में clean करना होगा  
  
    try:  
        # Video से audio निकालना और temp file में save करना  
        clip = VideoFileClip(self.video_path)  
        temp_audio = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")  
        temp_audio_path = temp_audio.name  
        temp_audio.close()  
  
        clip.audio.write_audiofile(temp_audio_path, verbose=False, logger=None)  
        clip.close()
```

```
self.progress['value'] = 30

self.root.update_idletasks()


# WAV file read करना

rate, data = wavfile.read(temp_audio_path)


# अगर audio stereo है, तो उसे mono में convert करें

if len(data.shape) == 2:

    data = data.mean(axis=1)


# Noise sample लेना (starting के 5000 samples)

noise_sample = data[:5000]


# Noise reduce करना

reduced_noise = nr.reduce_noise(y=data, sr=rate, y_noise=noise_sample)


# Reduced audio को temp file में save करना

cleaned_audio_file = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")

wavfile.write(cleaned_audio_file.name, rate, reduced_noise.astype("int16"))


# Cleaned audio path save करना

self.cleaned_audio_path = cleaned_audio_file.name


self.progress['value'] = 70

self.root.update_idletasks()
```



```
# Output video का नाम बनाना

output_video_path = os.path.splitext(self.video_path)[0] + "_cleaned.mp4"


# FFmpeg command run करना (new audio के साथ video combine करना)

ffmpeg_path = r"C:\ffmpeg\bin\ffmpeg.exe"

command = [

    ffmpeg_path, "-y",

    "-i", self.video_path,

    "-i", cleaned_audio_file.name,

    "-c:v", "copy",

    "-map", "0:v:0",

    "-map", "1:a:0",

    output_video_path

]


# Command को run करना और error check करना

result = subprocess.run(command, capture_output=True, text=True)

if result.returncode != 0:

    print("FFmpeg Error:", result.stderr)


self.progress['value'] = 100

self.root.update_idletasks()


messagebox.showinfo("Success", f"Cleaned video saved to:\n{output_video_path}")
```

```
except Exception as e:
```

```
    messagebox.showerror("Error", str(e))
```

```
finally:
```

```
    # Temporary audio file delete करना
```

```
    if temp_audio_path and os.path.exists(temp_audio_path):
```

```
        try:
```

```
            os.unlink(temp_audio_path)
```

```
        except Exception as cleanup_error:
```

```
            print(f"Cleanup error: {cleanup_error}")
```

```
# Program की entry point - GUI app चलाना
```

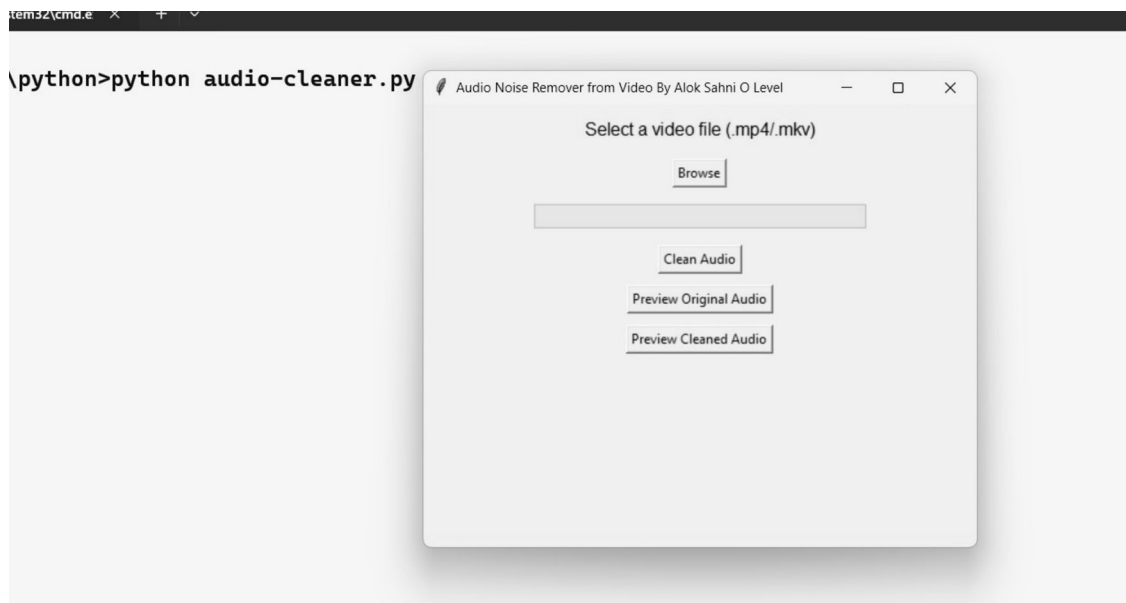
```
if __name__ == "__main__":
```

```
    root = tk.Tk()
```

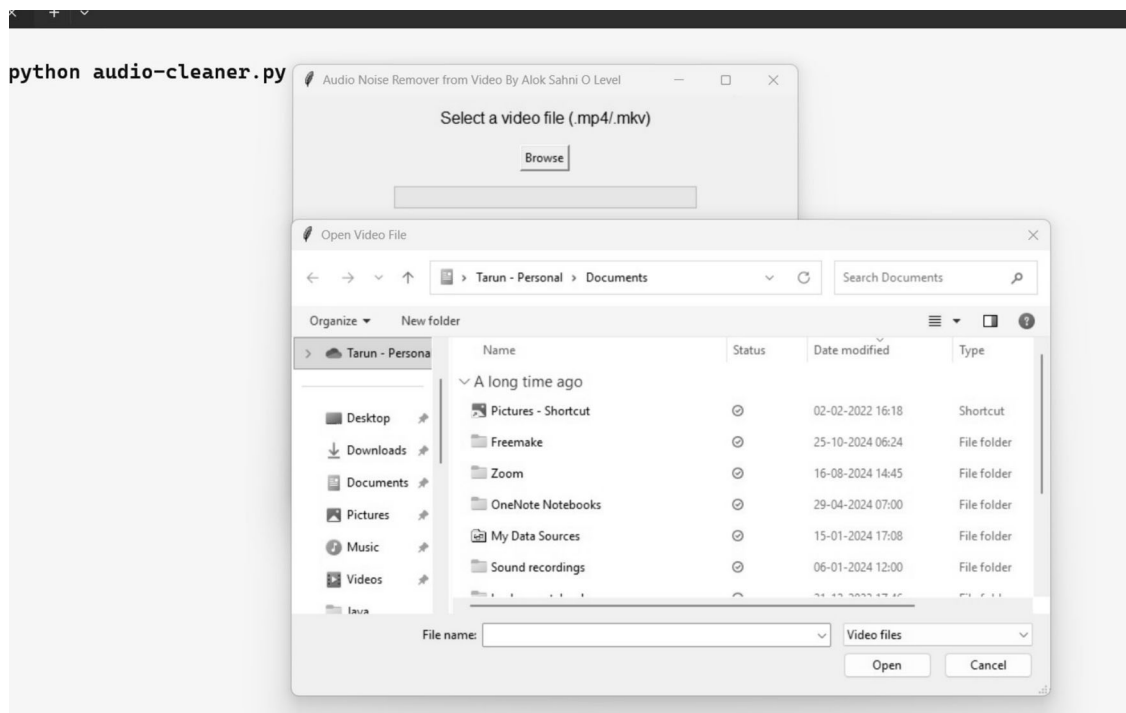
```
    app = NoiseRemoverApp(root)
```

```
    root.mainloop()
```

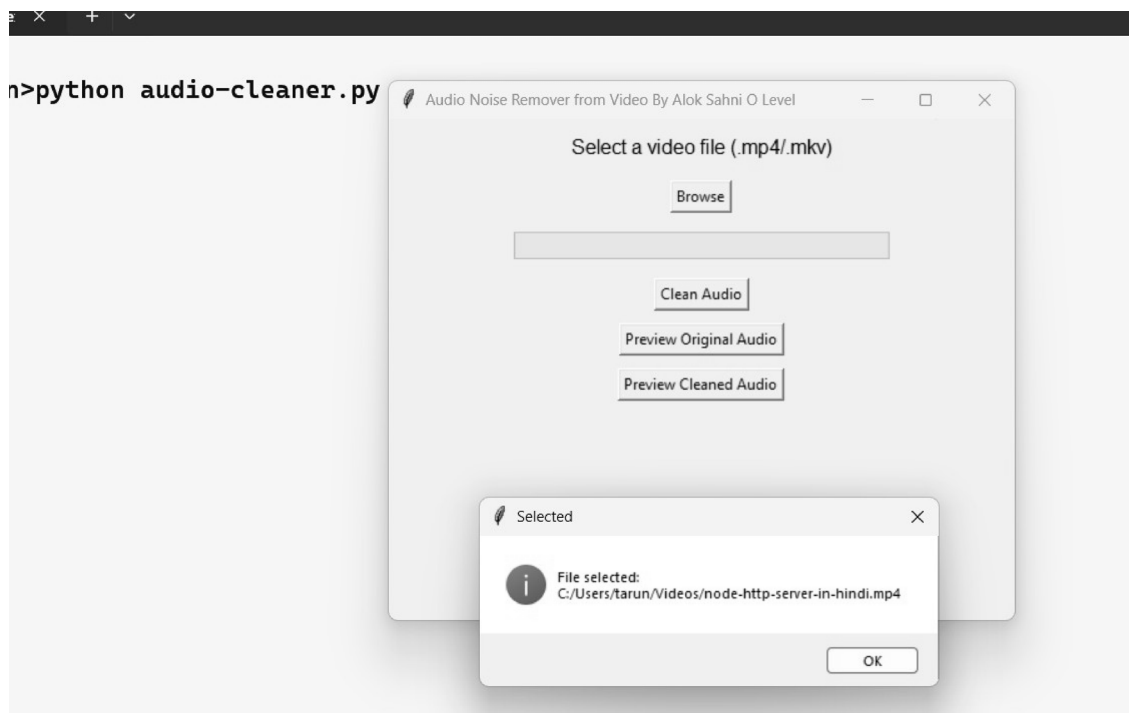
6. Screenshots



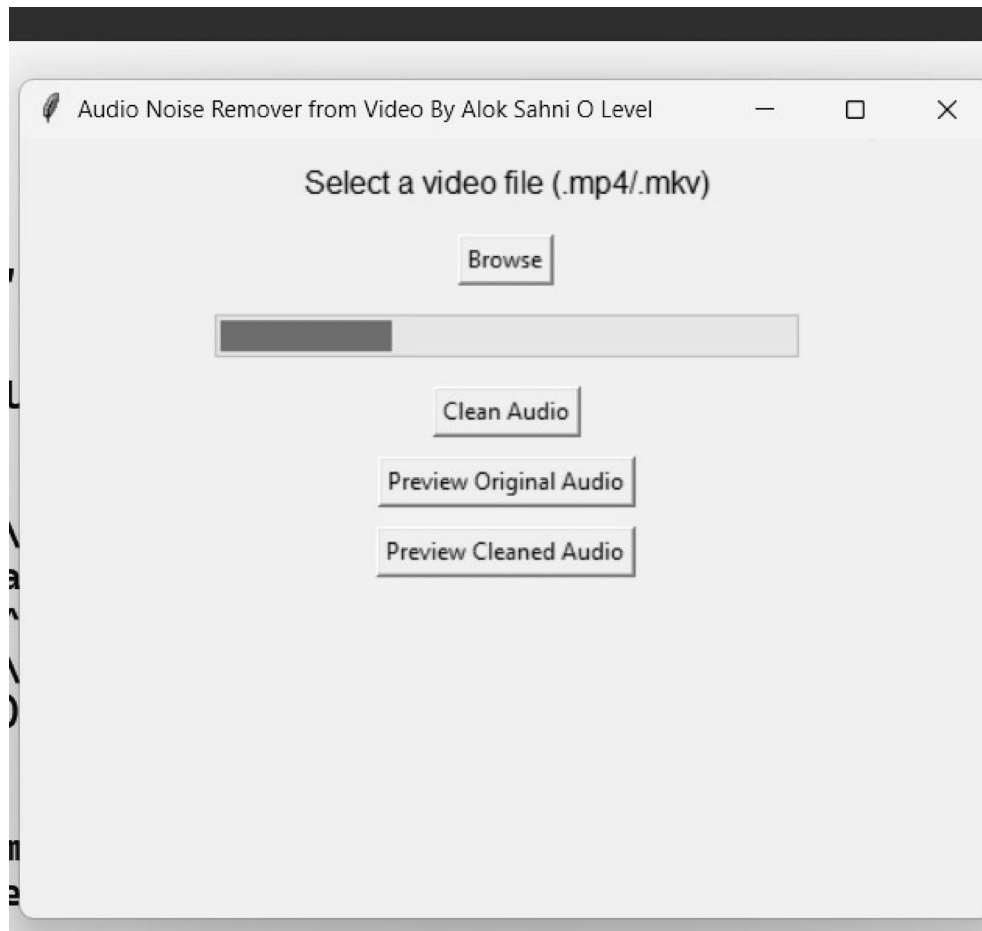
Main Screen



When Click on Browse



After Selecting a File



After clicking Clean Audio Button

7. Output

- Original and cleaned audio can be previewed.
- Cleaned audio is saved as a temporary .wav file.
- Final output is a new video file (e.g., original_cleaned.mp4) where the original noisy audio is replaced with noise-reduced audio.

Example:

Input: myinterview.mp4

Output: myinterview_cleaned.mp4

8. Conclusion

This project effectively demonstrates how Python can be used in media processing and GUI development. It bridges the gap between audio signal processing and practical usage with tools like FFmpeg. By automating the process of cleaning audio and embedding it back into the video, this tool can be a helpful utility for educators, content creators, editors, and students who want to improve the audio quality of their recorded material.

Bibliography

1. McKinney, W. (2017). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython. O'Reilly Media.
2. Lutz, M. (2013). Learning Python (5th Edition). O'Reilly Media.
3. VanderPlas, J. (2016). Python Data Science Handbook. O'Reilly Media.
4. Ascher, D., & Lutz, M. (2001). Learning Python. O'Reilly Media.
5. Reitz, K., & Schlusser, T. (2016). The Hitchhiker's Guide to Python: Best Practices for Development. O'Reilly Media.
6. Downey, A. (2015). Think Python: How to Think Like a Computer Scientist (2nd ed.). O'Reilly Media.
7. van Rossum, G., & Drake, F. L. (2009). Python 3 Reference Manual. Python Software Foundation.
8. Python Software Foundation. (2024). Python Official Documentation. Retrieved from <https://docs.python.org/3/>
9. FFmpeg Developers. (2024). FFmpeg Documentation. Retrieved from <https://ffmpeg.org/documentation.html>
10. MoviePy Developers. (2024). MoviePy Documentation. Retrieved from <https://zulko.github.io/moviepy/>
11. Scipy Community. (2024). SciPy Documentation. Retrieved from <https://docs.scipy.org/doc/scipy/>
12. Noisereducer Developers. (2024). noisereducer Library Documentation. Retrieved from <https://github.com/timsainb/noisereducer>
13. Tkinter Documentation. (2024). Python Tkinter GUI Library Guide. Retrieved from <https://docs.python.org/3/library/tkinter.html>
14. Playsound Developers. (2024). Playsound Library for Python. Retrieved from <https://pypi.org/project/playsound/>
15. OpenAI. (2023). Applications of AI in Audio and Video Processing. OpenAI Blog.
16. Verma, T., & Sahni, A. (2025). Audio Noise Remover from Video: A Python-Based GUI Application. Unpublished student project.
17. Smith, J. O. (2010). Introduction to Digital Filters with Audio Applications. W3K Publishing.
18. Mitra, S. K. (2010). Digital Signal Processing: A Computer-Based Approach. McGraw Hill.
19. Lyons, R. G. (2011). Understanding Digital Signal Processing (3rd Edition). Pearson Education.

20. Audacity Team. (2024). Noise Reduction Techniques in Audio Processing.
Retrieved from <https://manual.audacityteam.org/>